the database to do the work. This has a consequence: Your custom columns can't participate in the sort, as they aren't in the database. The Birth Weekday column created in the previous example is not clickable, as you can't sort by it.

The Django Admin also supports search. To enable this feature, you add a `search_fields` attribute to the `admin.ModelAdmin` class. Like `list_display`, this takes an iterable of field names—in this case, the fields to matched against a search term. When `search_fields` is present, the Django Admin adds a search box to the top of the page.

Different search behavior can be specified by adding a suffix to the name of the participating field. Without a suffix, the search term gets used as a substring for matches. For example, using `last_name` in `search_fields` and searching for *e*, results in listing `Hendrix` and `Lennon` because they have the letter *e* in their last names.

The `__startswith` suffix causes search terms to match the beginning of the field. Configuring `first_name__startswith` and searching for *j* returns `Jimi Hendrix`, `John Bonham`, and `John Lennon`. The `__exact` suffix requires search terms to be exact matches (case insensitive).

The `search_fields` attribute can take multiple fields, and you can mix and match the suffixes as you like within the values. The following listing modifies the `list_display` attribute of `MusicianAdmin` to add the `first_name` field as a column and makes the first and last names searchable.

Listing 5.3 Modified `MusicianAdmin` class to support search

```
# RiffMates/bands/admin.py
...
class MusicianAdmin(admin.ModelAdmin):
    list_display = ("id", "last_name", "first_name", "show_weekday", )
    search_fields = ("last_name", "first_name", )
    ...
```

5.2.2 Filtering

In addition to searching, you can filter content as well. The way you implement this is similar: you add the `list_filter` attribute to your `admin.ModelAdmin` class. The simplest form of `list_filter` is the name of a field to filter by.

Filters are shown as widgets on the right-hand-side of the listing page. For fields in the `list_filter` configuration, all values for the field are shown as filter links—which can be problematic.

For a field that has a limited selection, like a `BooleanField` or anything restricted using the `choices` attribute, a complete list of values is fine. For a wide-open field, like `last_name`, this becomes both ugly and a performance limitation. For a large dataset, you do not want every single last name in your database showing on the right-hand side; in this case, you should use search capabilities instead.

The default filter for a `DateField` shows a small set of values to choose from: today, the last seven days, this month, and this year. For our musicians' birthdays, this isn't